NestSelf authentication and authorization: quick guide

The ten-minute version: what the auth platform does, the decisions behind it and where to look. The detailed walkthrough is `implementation.md`; the full specification with every requirement is `docs/tasks/auth/auth-strategy.md`.

What it does

NestSelf serves five populations from one platform: employees of client companies, their HR and managers, independent experts, NestSelf staff and external suppliers. Each person has exactly one identity for life, may belong to several employers over time, and sees only what their current role allows. Authentication answers "who is this"; authorization answers "what may they do right now, in this organization".

The decisions

Decision	What we chose	Why
Identity provider	Amazon Cognito, one user pool per environment in Frankfurt (eu-central-1)	Email one-time codes, authenticator apps and passkeys without building them; free at pilot scale; EU data residency. Behind an interface so it can be replaced.
Sign-in methods	Employees use an email code. Experts and staff use a password plus an authenticator app. Employers may later bring their own SSO.	Removes passwords for the largest population; no SMS costs in Serbia and Hungary.
Web sessions	The browser gets an opaque cookie; the API keeps Cognito tokens encrypted in Redis (Backend-for-Frontend)	Tokens never reach JavaScript; revoking a session is deleting a key; CSRF handled with a double-submit token.
Mobile	The app talks to Cognito directly (PKCE) and calls the API with a bearer token	The standard native pattern; tokens live in the device's secure store.
Authorization	Hand-built grants evaluated by a pure `authorize()` function	Delegation, spending ceilings, effective-access previews and a 150-case golden test table are native; tokens carry no roles, so access changes apply within seconds.
Organization structure	A tree of org units with a closure table	Cheap ancestor queries for "manager of this team and everything under it".
Database access	Drizzle ORM with PostgreSQL row-level security	Tenant isolation lives in the database: a query without an organization filter returns nothing. Partial indexes, checks and views are part of the schema.
Audit	Append-only, partitioned `audit_events` with retention tiers	Every consequential action is recorded; nothing sensitive (tokens, message bodies, raw IPs) ever is.
Background work	BullMQ on Redis, run by a separate worker process	Reconciliation (expired grants, stale mobile token families, audit partitions, expired sessions, expired emergency access) is audited like any user action and never runs inside the API process.

Decision	What we chose	Why
Step-up	Sensitive actions need a sign-in or a fresh email code from the last 5 to 30 minutes, depending on the action's class	Granting access, ending memberships, reading confidential records and emergency access stay safe on an unattended screen. Every class uses an email code for now; the authenticator for the emergency class is an owner decision.
Legacy roles	Removed from the database; every permission is a grant	One source of truth. `db:backfill-grants` converted the old role columns before migration `0005` dropped them.
Organization ownership	Exactly one `employer_owner` per organization: the organization's superadmin, who grants and revokes access and can hand the role to another member. NestSelf staff can change anything for an organization, including transferring its ownership	One accountable person per client, support never locked out. A second owner is refused with 409 `owner_exists`.
Enforcement	Every route declares `@Public`, `@Authenticated` or `@Authorize`; a CI test fails otherwise	Default deny cannot be forgotten.
Rollout	Cognito first, platform stacks later, mobile last	Local development stays local; the CDK app deploys only the auth stack unless asked for more.

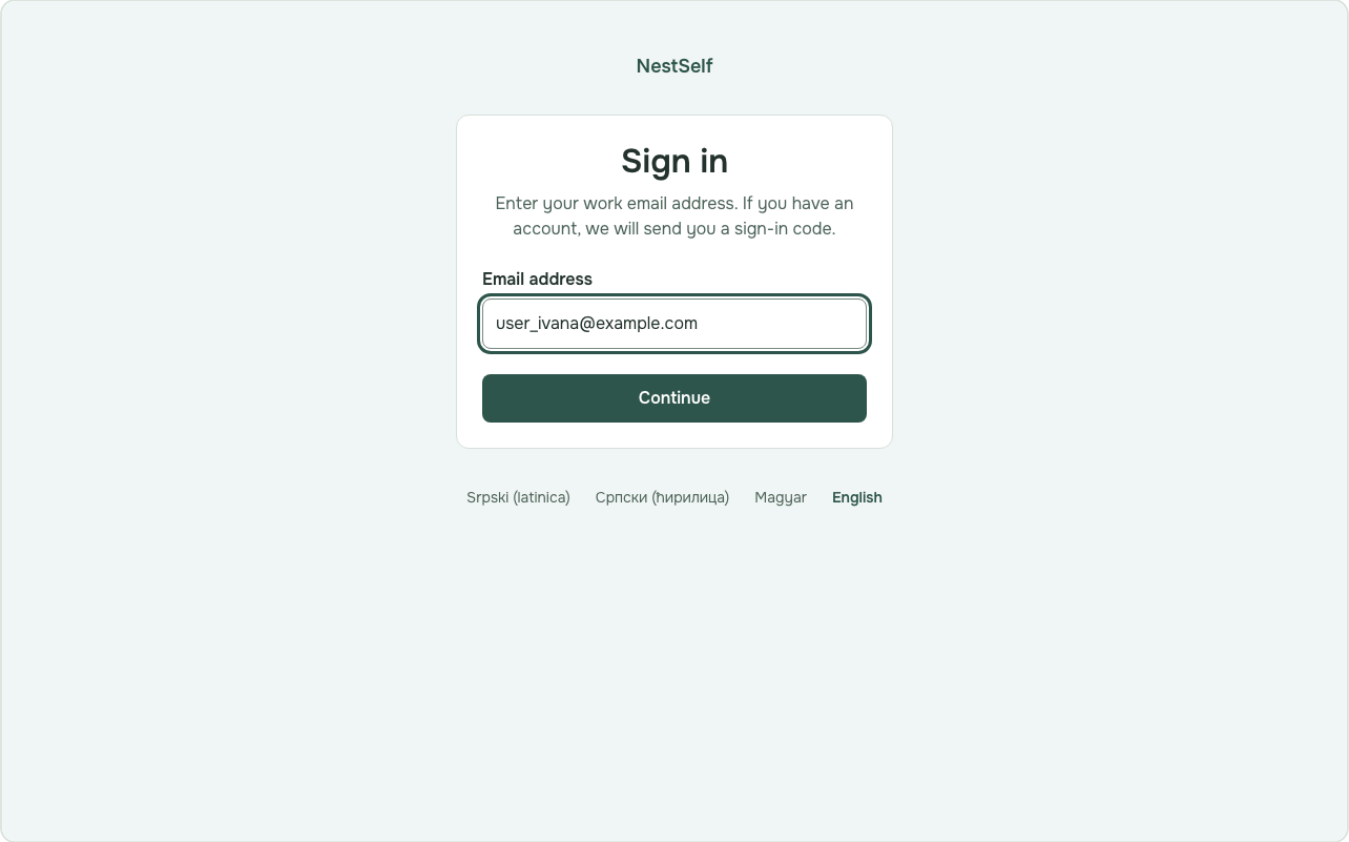
How a request works

1. The browser sends the session cookie (or the app sends a bearer token).
2. `AuthenticationGuard` turns it into a `Principal`: who they are, their memberships, their active context.
3. `CsrfGuard` checks the CSRF header on mutating browser requests.
4. `AuthorizationGuard` reads the route's `@Authorize(action, resolver)`, loads grants and assignments (cached in Redis for 30 to 60 seconds), and asks `authorize()`. Denied requests get 404 when the caller should not even know the record exists, 403 otherwise. For a sensitive action it also checks that the caller authenticated recently enough and answers 403 `step_up_required` if not.
5. The handler runs its writes inside an organization-scoped transaction; PostgreSQL row-level security blocks anything outside that organization.
6. Consequential actions are written to the audit log; access changes publish an invalidation that drops caches and disconnects affected WebSocket clients.

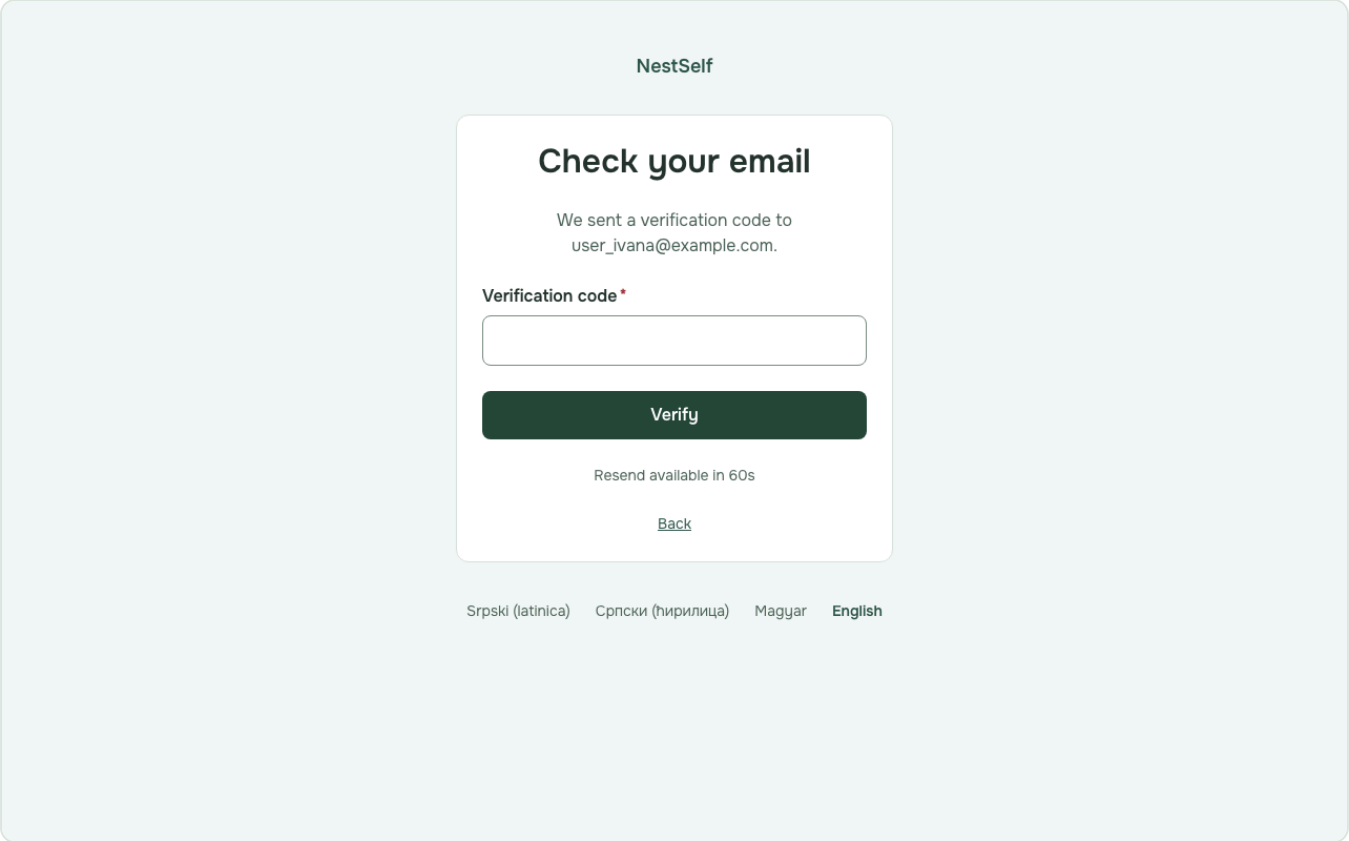
How sign-in works

- **Employee**: enter email → receive an **eight digit** code (Cognito's length, not configurable) → enter it → session cookie. A wrong code can be retyped on the same challenge up to three times; after the third the user requests a new code, and five wrong codes in total lock the address for 15 minutes (`implementation.md`, section 6). The code email is written in the language of the sign-in screen.
- **Expert or staff**: enter email → password → authenticator code (first time: scan a QR code) → session cookie. Invited staff set a password first.

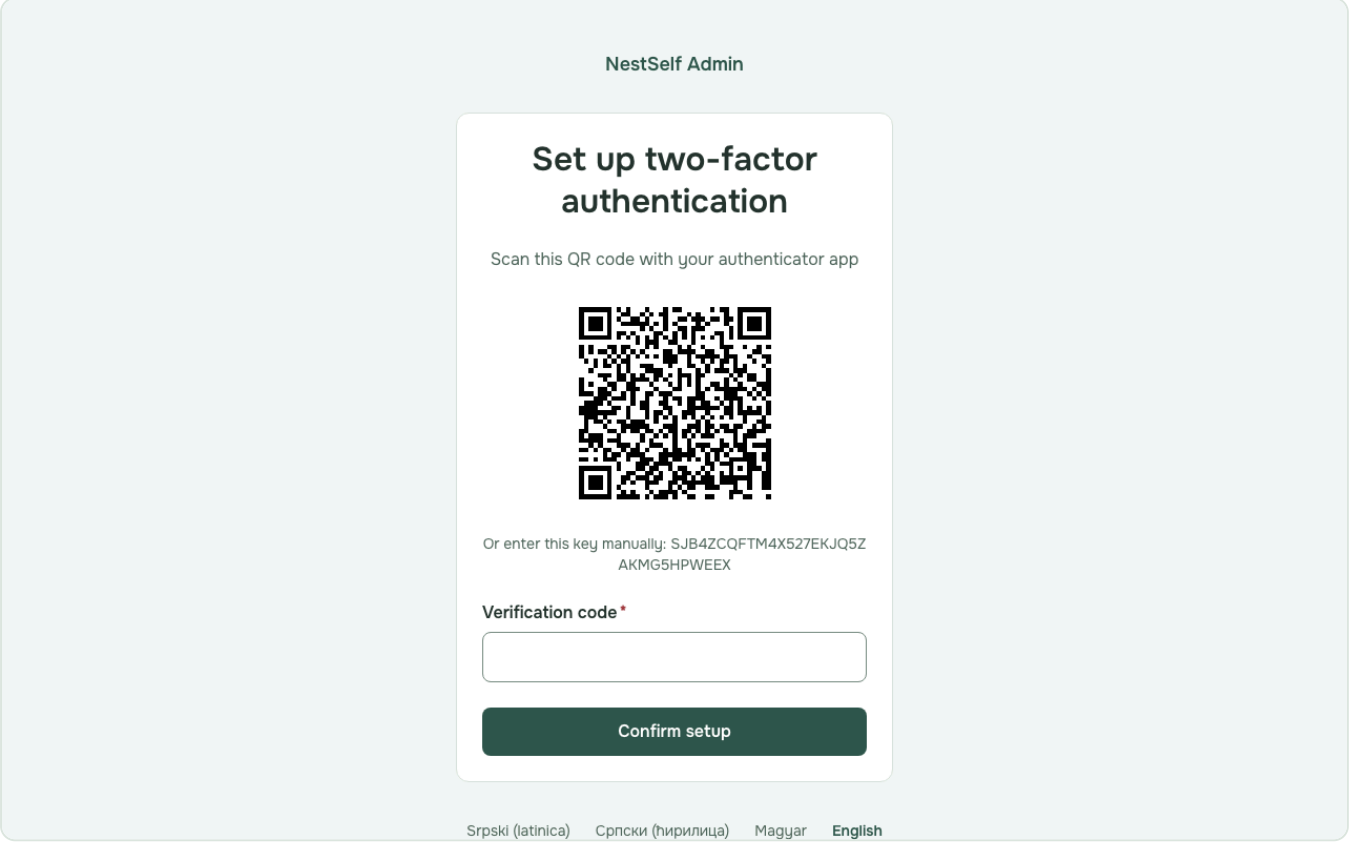
- **Invitation**: HR invites by email; the link previews the organization name and activates the membership on acceptance, then runs the matching sign-in.
- **Mobile**: email code directly against Cognito; the app refreshes tokens itself and uses short-lived tickets to open WebSockets.



Workspace sign-in: one field first; the address decides the method.



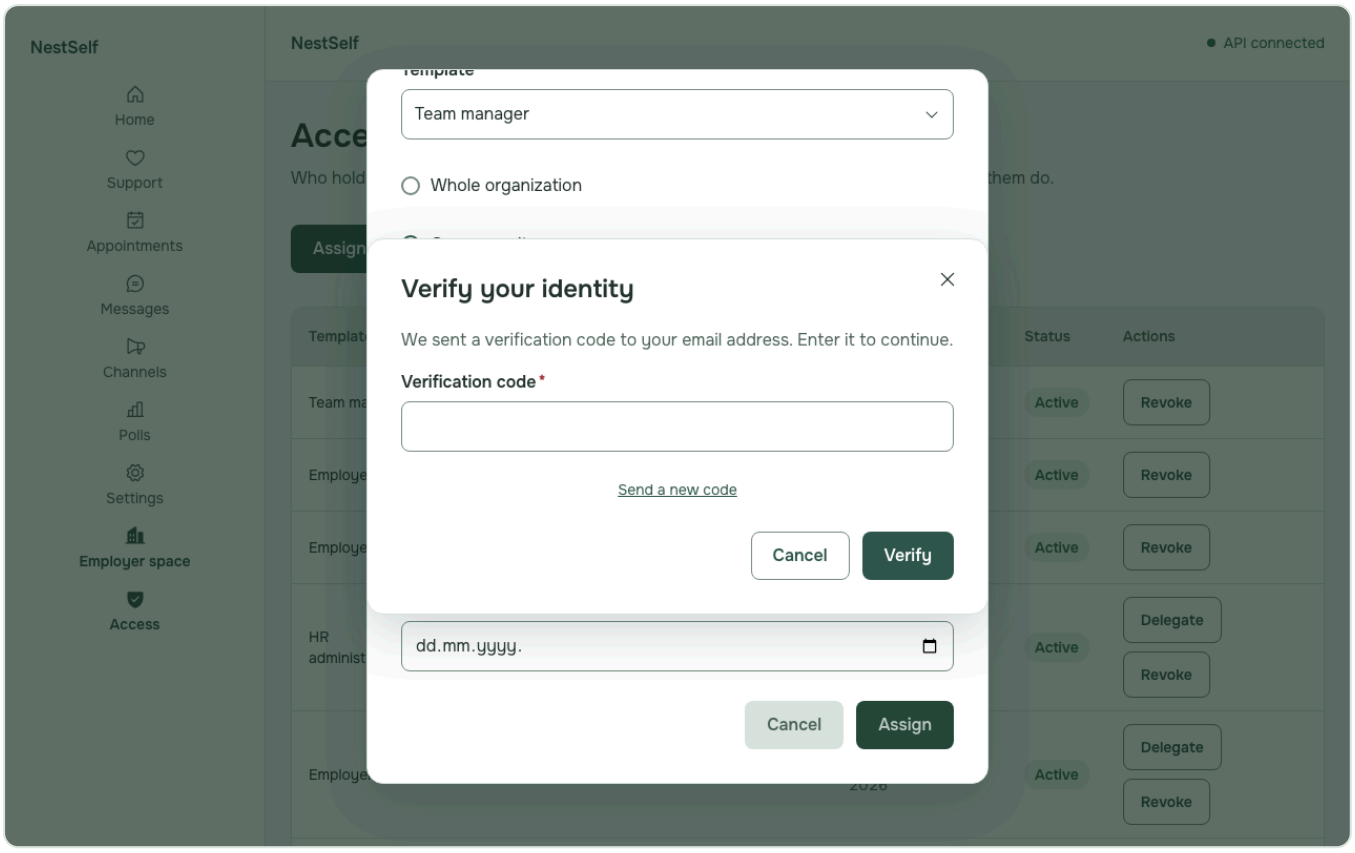
Employees then type the code from the email.



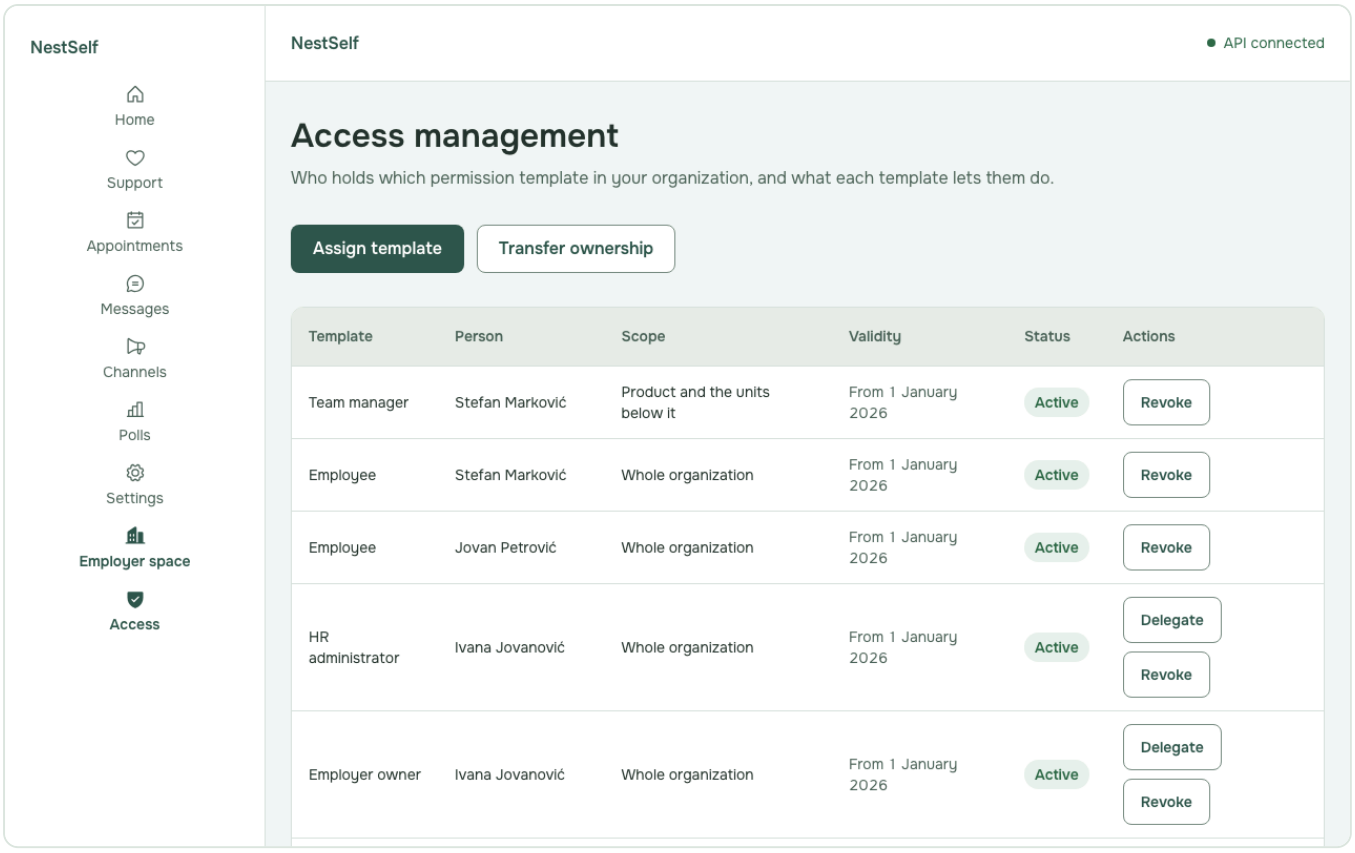
Staff and experts enrol an authenticator on their first sign-in to the admin app.

Step-up and the access screens

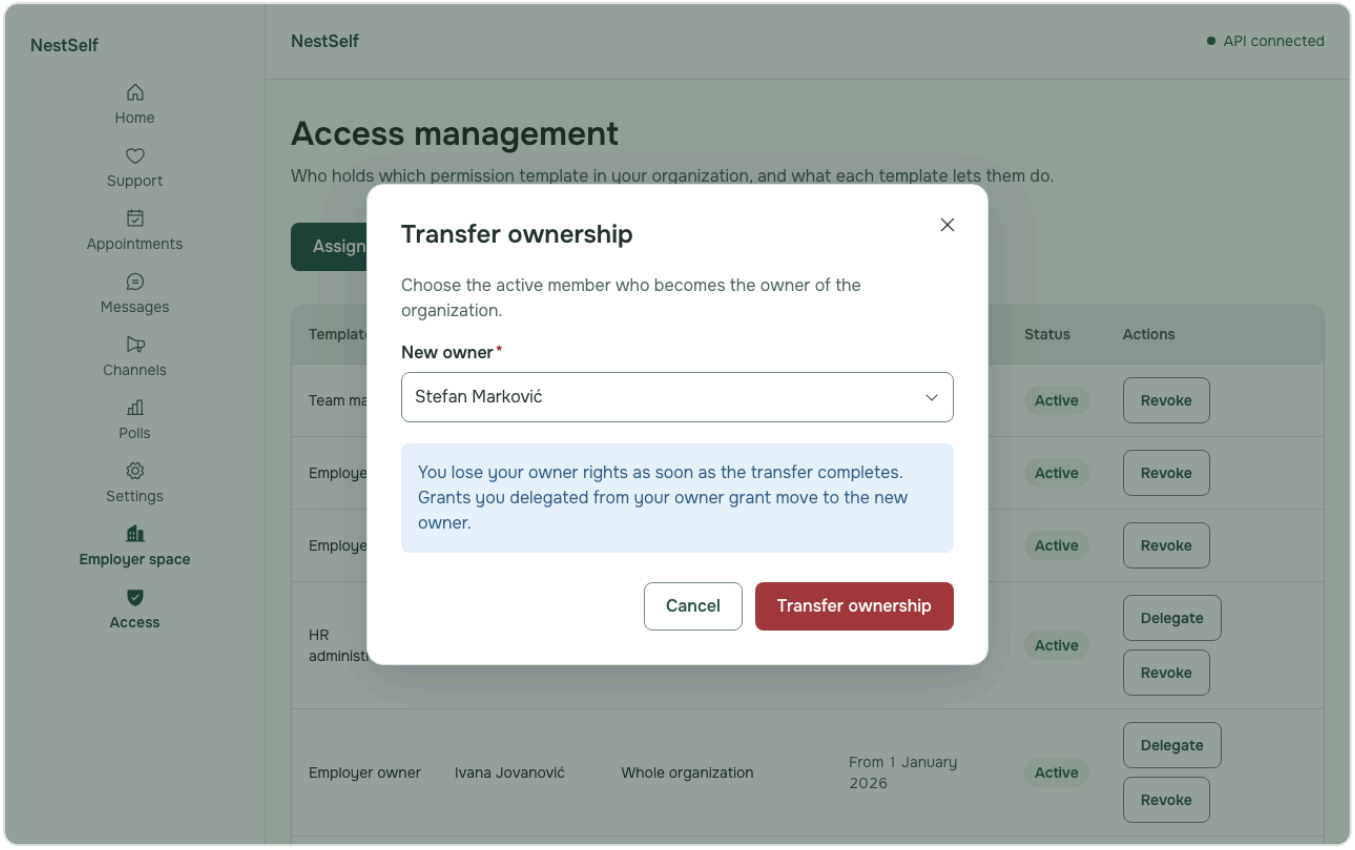
- **Step-up**: when the API answers `step_up_required`, the web apps open a dialog, email a fresh code and retry the action once the code is accepted. Cancelling leaves the action undone without an error. Wrong codes follow the same three-tries rule as sign-in. Mobile has no step-up screen yet.
- **Workspace, `/hr/access`**: for the organization's owner (the sample Ivana, who is also its HR administrator). Shows the organization's grants, previews any member's effective access, assigns and revokes templates, delegates part of one's own grant and transfers ownership to another active member.
- **Admin, organization Access tab and `/access`**: for platform owners. Assigns organization templates to an organization's members (organization-wide; org-unit grants are HR's job), transfers an organization's ownership (also when it has no owner yet), assigns staff templates, revokes grants and looks up any user's access per context.
- **Sample mode** shows the same screens over fixtures with every change disabled.



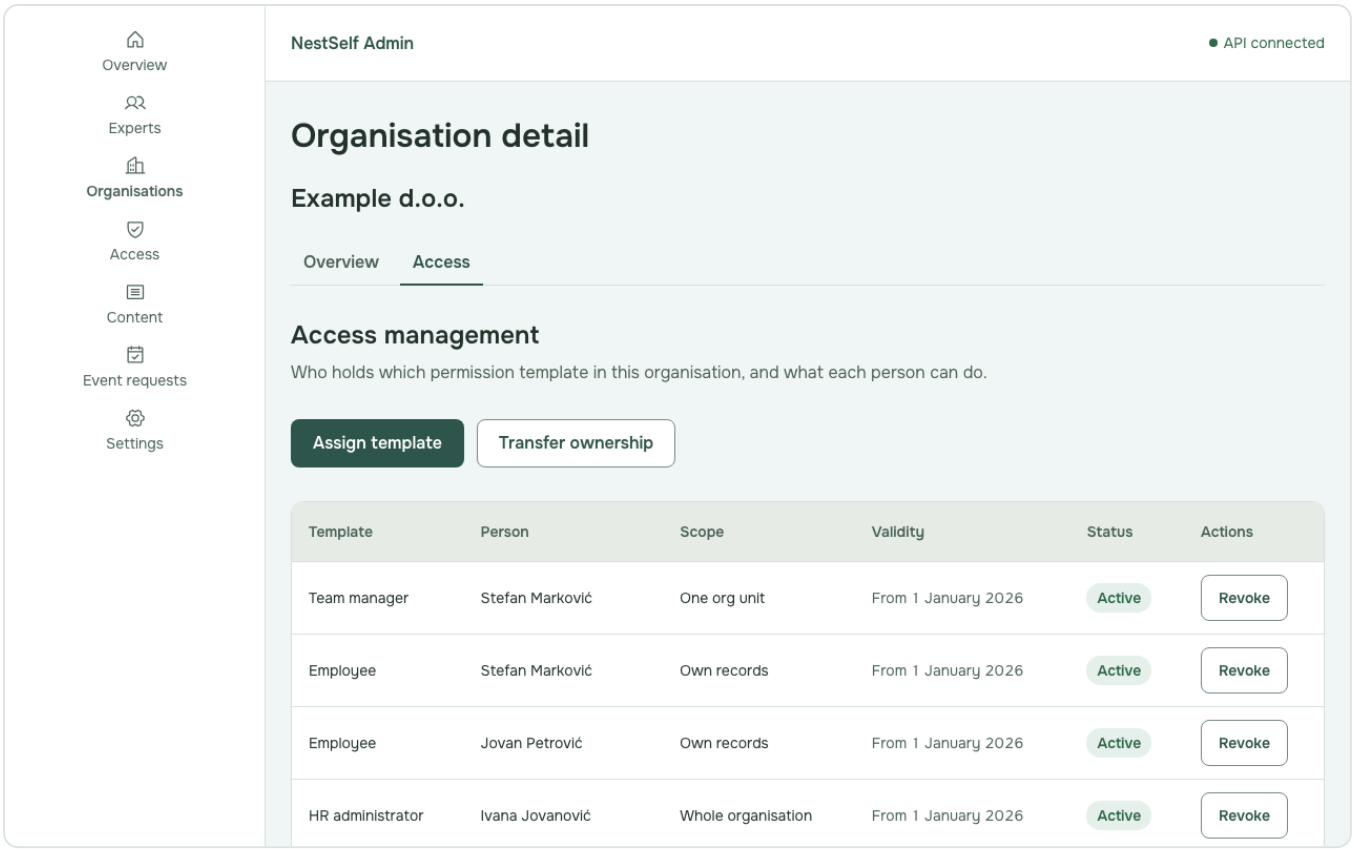
Step-up: a fresh email code before a sensitive change, here an assignment.



Workspace `/hr/access` for the owner.



Transfer ownership: only the current owner sees it; the old owner's delegations move to the new owner.



The same organization seen by staff in the admin app.

The screenshots are of the real screens against the stub API with the fictional sample people; `pnpm docs:auth:screenshots` regenerates them (`implementation.md`, section 19).

Where to look

Question	Place
Which routes exist and which decorator they carry	`apps/api/docs/auth-core.md`
The guards and sign-in services	`apps/api/src/auth/`, `apps/api/src/authz/`
The decision engine and its golden table	`apps/api/src/authz/authorize.ts`, `golden-table.json`
Actions, templates, shared schemas	`packages/contracts/src/`
Database schema, migrations, seed	`apps/api/src/db/schema/`, `apps/api/drizzle/`, `apps/api/src/db/seed/`
Sessions and Redis keys	`apps/api/src/session/`, `apps/api/src/redis/redis-keys.ts`
Sign-in screens	`apps/workspace/src/auth/screens/`, `apps/admin/src/auth/screens/`, `apps/mobile/src/auth/`
Cognito infrastructure and rollout steps	`CDK-setup.md` (repo root), `infra/cdk/lib/auth-stack.ts`, `infra/cdk/README.md`

Question	Place
Nightly jobs and the worker process	`apps/api/src/jobs/`, `apps/api/src/worker/main.ts`
Grant administration, confidential reads, emergency access	`apps/api/src/grants/`, `apps/api/src/professional/`, `apps/api/src/emergency/`
Step-up	`apps/api/src/auth/step-up.service.ts`, `apps/api/src/authz/step-up-freshness.ts`, the web apps' `src/auth/step-up-dialog.tsx`
Access screens	`apps/workspace/src/features/access/`, `apps/admin/src/features/access/`
Phase 2 status and what is left	`docs/tasks/auth/phase-2-readiness.md`, `implementation.md` section 21
Environment variables	`docs/environment.md`, `apps/api/src/config/env.ts`

Running it locally

```
pnpm install && pnpm db:up
cp apps/api/.env.example apps/api/.env
pnpm --filter @nestself/api db:migrate && pnpm --filter @nestself/api db:seed
pnpm --filter @nestself/api dev
pnpm --filter @nestself/api worker:dev   # optional: jobs, needs REDIS_URL (runs through the
Nest compiler)
```

Stub mode uses a fake identity provider pre-loaded with the sample people. Employees (`user_jovan@example.com`, `user_ivana@example.com`, `user_stefan@example.com`, `user_dragan@example.com`) sign in with the code printed in the API log, and Ivana reaches `/hr/access`; staff and experts (`user_nikola@example.com`, `user_milica@example.com`) use the password `Nestself-dev-1234!` and enrol an authenticator on first sign-in. Set `VITE_DATA_SOURCE=api` in a web app's `.env.local` to use the real screens. Skip `pnpm db:up` when PostgreSQL and Redis already run outside Docker; the Compose file claims port 6379 and fails against a local Redis or Valkey.

A real Cognito dev pool exists since 2026-10-24 (`eu-central-1_g6OGYONQ0` in account `634972095709`). Set `AUTH_MODE=cognito` with the values `infra/cdk/scripts/env-from-outputs.mjs dev` prints, and note that pool usernames are email addresses, not handles. `implementation.md` section 17.1 has the deployed coordinates and the three traps worth knowing before touching it.

Glossary

- **Principal**: the resolved identity for a request (user, memberships, active context, how they authenticated).
- **Context**: the hat the user wears right now: personal, a specific organization, expert, or operations.
- **Grant**: a user's permission to perform actions within a scope, with conditions and a validity window.
- **Template**: a named starting set of actions (`hr_admin`, `team_manager`, `employee`, ...) grants are created from.
- **Assignment**: a link to a work item (case, queue, client) that confers access without a grant.

- **Closure table**: every ancestor to descendant pair of the org tree, so scope checks are a set lookup.
- **Step-up**: re-authenticating recently before a sensitive action; enforced by the API.
- **Delegation**: handing part of one's own grant to another member, at most three levels deep and never beyond the parent grant.

What is not done yet

Enterprise SSO (phase 3), mobile step-up, the remaining nightly jobs (Cognito user export, retention sweeper, audit export), file and export authorization, the security baseline that waits for staging, and the compliance track (privacy notices, processing agreement, breach procedure), which must land before the first real employee signs in. Screens for members and invitations, org units, approval authorities, emergency access, sessions and suspension come next. Open owner decisions: the authenticator for emergency step-up and a `cases` table before anything writes conversations. Who administers grants inside an organization was decided on 2026-10-24: the single employer owner, and NestSelf staff. The full list is `implementation.md` section 21; the status per task is `docs/tasks/auth/phase-2-readiness.md`.